発展プログラミング (Advanced Programming)

3. デザインパターンの基礎

尾崎 知伸 ＜ozaki.tomonobu@nihon-u.ac.jp＞

Singleton パターン
～たった一つのインスタンス～

Singleton パターン

- Singletonパターン：インスタンスが一つしか存在しない（生成されない）ことを保証するパターン
  - 単一のクラスからなる非常に単純なパターン
  - コンストラクタのカプセル化＋（自身による）静的インスタンスの生成・公開
- 利用シーン
  - カウンタ（例：通し番号），状態管理変数（例：システムの状態），リソースへのアクセス（例：NWインタフェースにアクセスするインスタンス）などを，アプリケーション全体を通して唯一のものとして扱いたい場合
  - 通し番号（ID番号）を入れたチケットの発券の例
    - 複数のチケットを生成するが，通し番号は一元管理したい

```
class Main {
    public static void main(String[] args) {
        TicketMaker tMaker1 = new TicketMaker();
        for(int i = 0; i < 5; i++) {
            System.out.println("No. " + tMaker1.getNextTicketNumber());
        }

        TicketMaker tMaker2 = new TicketMaker();
        for(int i = 0; i < 3; i++) {
            System.out.println("No. " + tMaker2.getNextTicketNumber());
        }
    }
}
```

実行結果

```
No. 1000
No. 1001
No. 1002
No. 1003
No. 1004
No. 1000
No. 1001
No. 1002
```

```
class TicketMaker {
    private int ticket = 1000;
    public int getNextTicketNumber() {
        return ticket++;
    }
}
```

TicketMakerを利用する際に，
そのインスタンスを何個でも自由に作成可能
→一つしか作らせないようにしたい！

サンプルプログラム

lec03_singleton/*.java

コンパイル方法: javac lec03_singleton/Main.java
実行方法: java lec03_singleton/Main

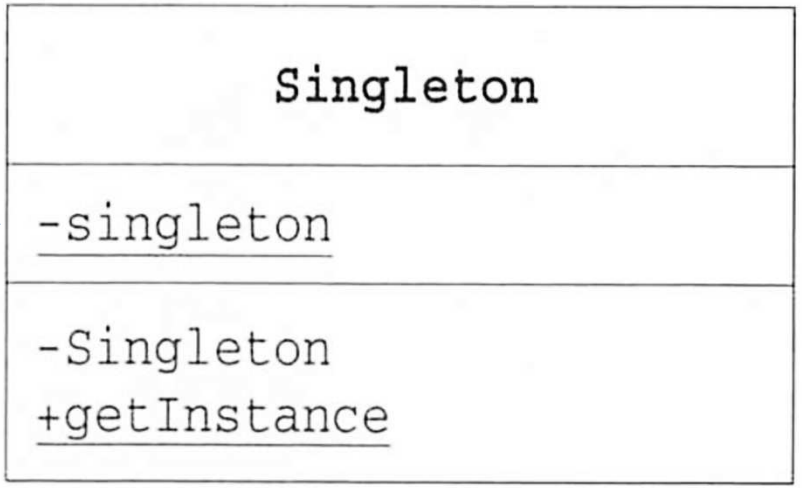
プログラム内容

- Singletonクラスのインスタンスを2回獲得する.
- 両者が同一オブジェクトであるか確認する.

- クラス図：「ー」はprivate, 「＋」はpublic, 下線はstatic
- コンストラクタがprivateであり，このクラスのインスタンスを取得するためにはgetInstance()を利用しないといけないのがポイント

このクラスのインスタンスが
private（外からアクセス不可）かつ
static（インスタンス変数ではなくクラス変数）

コンストラクタがprivate



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

全体を通して唯一で（static）
外部からアクセスできない（private）
Singletonクラスインスタンスを作成

```java
public class Singleton {
    private static Singleton singleton = new Singleton();

    private Singleton() {
        System.out.println("インスタンスを生成しました");
    }

    public static Singleton getInstance() {
        return singleton;
    }
}
```

Singletonクラスのインスタンスを
取得するためにはこのメソッドを利用する他ない

```java
public class Main {
    public static void main(String[]args) {
        System.out.println("Start.");

        Singleton obj1 = Singleton.getInstance();
        Singleton obj2 = Singleton.getInstance();

        if( obj1 == obj2 ) {
            System.out.println("obj1とobj2は同じインスタンスです");
        }else {
            System.out.println("obj1とobj2は同じインスタンスではありません");
        }
        System.out.println("End.");
    }
}
```

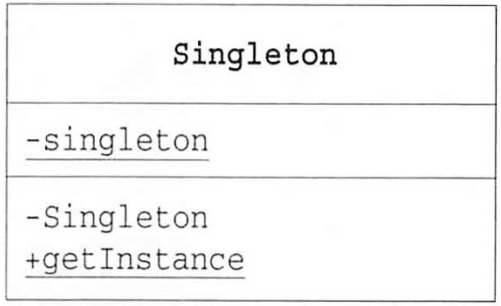
実行結果

```
Start.
インスタンスを生成しました
obj1とobj2は同じインスタンスです
End.
```

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Singletonパターンの登場人物と考え方を広げるヒント

- Singletonの役
  - コンストラクタのprivate化 → 外部からの生成の禁止
  - 静的なインスタンスの生成と保持
  - インスタンスを得るためのstaticメソッドを公開
  - このメソッドはいつも同じインスタンスを返却



- なぜ制限を設ける必要があるのか？
  - インスタンスが複数存在すると，相互に影響しあって思いがけないバグを生み出す可能性がある
  - インスタンスが一つしかないという保証があれば，その前提条件下でのプログラミングが可能

- インスタンス生成のタイミング
  - 前述のプログラムでは"start. →インスタンスを生成しました"の順に表示
  - getInstanceメソッドの初回呼び出し時に，Singletonクラスが初期化される
    - このタイミングでstaticフィールドの初期化＝インスタンス生成が実行

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

演習1

- lec03_singleton/*.java
- TicketMakerクラス(TicketMaker.java)にSingletonパターンを適用し，チケットが通し番号になることが保証されるように修正せよ
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec03_singleton;

- （余力のある学生さん向け）インスタンスの個数が3個に限定されているクラスTripleを作成せよ．各インスタンスには，0, 1, 2 という番号がついており，getInstance(int id) でid番目のインスタンスが得られるものとする

- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

Builder パターン
～複雑なインスタンスを組み立てる～

Builder パターン

- 複雑なインスタンスを組み立てる
  - 複雑な構造を持ったものを作り上げるとき，一気に完成させることは困難
  - まず全体を構成している各部分を作り，段階を踏んで組み上げる

- Builderパターン：構造を持ったインスタンスを組み上げていくパターン
  - 複雑な構造のインスタンスの作成を（外部に対し）隠ぺいする
    - 利用者は，インスタンス生成をお願いする相手（Director役）が分かれば十分で，内部におけるインスタンス生成過程を知る必要はない

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

サンプルプログラム

lec03_builder/*.java

コンパイル方法: javac lec03_builder/Main.java
実行方法:　　java lec03_builder/Main { plain, html}

利用シーン

- 下記の構造の文書の作成
  - タイトルを一つ含む
  - 文字列をいくつか含む
  - 箇条書き項目をいくつか含む
- 文書の具体的なフォーマットは，プレーンテキストとHTML
  - 実行時に指定する

プログラムを使って
この構造を持った文書を
作成することを考える

実行例：java lec03_builder/Main text

```
==============================
『Greeting』
■朝から昼にかけて

  ・おはようございます.
  ・こんにちは.

■夜に

  ・こんばんは.
  ・おやすみなさい.
  ・さようなら.

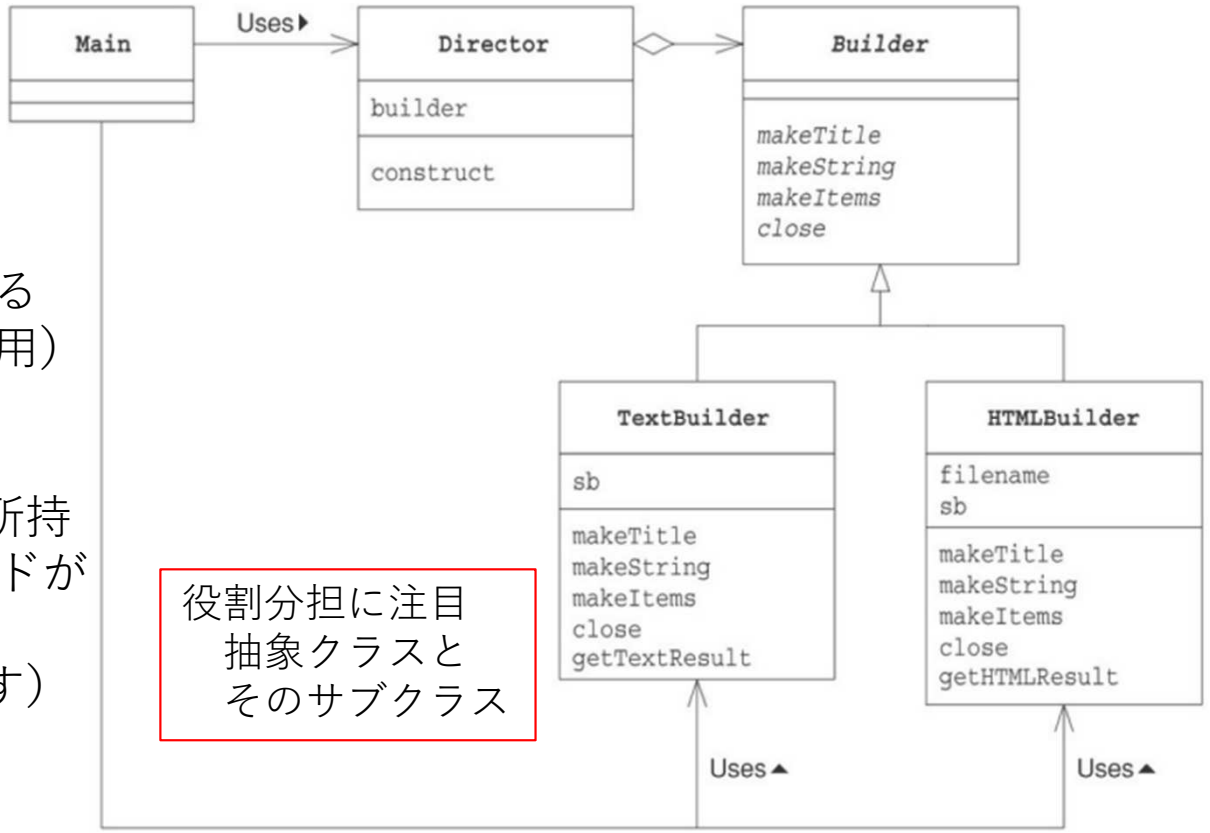
==============================
```

実行例：java lec03_builder/Main html
（Greeting.htmlが作成される）

```
<html><head><title>Greeting</title></head><body>
<h1>Greeting</h1>
<p>朝から昼にかけて</p>
<ul>
<li>おはようございます．</li>
<li>こんにちは．</li>
</ul>
<p>夜に</p>
<ul>
<li>こんばんは．</li>
<li>おやすみなさい．</li>
<li>さようなら．</li>
</ul>
</body></html>
```

- Builderクラス：
  - 文書を構成するための
    メソッドを定める
    （抽象クラス）
- Directorクラス：
  - 定められたメソッドを使って
    具体的な文書を作る
  - Builderインスタンスを利用

名　前	解　説
Builder	文書を構成するためのメソッドを定めた抽象クラス
Director	1つの文書を作るクラス
TextBuilder	プレーンテキスト（普通の文字列）を使って文書を作るクラス
HTMLBuilder	HTMLファイルを使って文書を作るクラス
Main	動作テスト用のクラス



Directorのインスタンスは，
Builderのインスタンスにいろいろ委譲する
（Builderインスタンスのメソッドを利用）

Template Methodパターンでは，
抽象クラスがテンプレートメソッドを所持
今回は，Directorが持つconstruct メソッドが
全体の処理の流れを決める
（＝テンプレートメソッドの役割を果たす）

タイトル，文字列，箇条書き項目を
適切な順番で必要数記載するという
複雑な作業をconstruct()内で行うことで
Mainクラス(=利用者)は複雑な処理を行わなくて済む

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Builder パターン

lec03_builder/*.java

```java
public class Director {
    private Builder builder;

    public Director(Builder builder) {
        this.builder = builder;
    }

    public void construct() {
        builder.makeTitle("Greeting");

        builder.makeString("朝から昼にかけて");
        builder.makeItems(new String[]{
                "おはようございます。",
                "こんにちは。",
        });

        builder.makeString("夜に");
        builder.makeItems(new String[]{
                "こんばんは。",
                "おやすみなさい。",
                "さようなら。",
        });

        builder.close();
    }
}
```

```java
public abstract class Builder {
    public abstract void makeTitle(String title);    //タイトルを構築
    public abstract void makeString(String str);     //文字列を構築
    public abstract void makeItems(String[] items); //箇条書きを構築
    public abstract void close();                    //文章を閉じる
}
```

Builderを継承：抽象メソッドを実装

```java
public class TextBuilder extends Builder {
    private StringBuffer buffer = new StringBuffer();

    public void makeTitle(String title) {
        buffer.append("==============================\n");
        buffer.append("『" + title + "』");
        buffer.append("\n");
    }
    public void makeString(String str) {
        buffer.append('■'+ str + "\n");
        buffer.append("\n");
    }
    public void makeItems(String[] items) {
        for (int i = 0; i < items.length; i++) {
            buffer.append("  ・" + items[i] + "\n");
        }
        buffer.append("\n");
    }
    public void close() {
        buffer.append("==============================\n");
    }
    public String getTextResult() {
        return buffer.toString();
    }
}
```

複雑な処理はここに隠ぺい
・テンプレートのように動作（手順）を決める
・各処理は，builderに移譲する

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

実行時の引数が格納される文字型配列　　lec03_builder/*.java

```
public class Main {
    public static void main(String[] args) {
        if (args.length != 1) {
            usage();
            System.exit(0);
        }
        if (args[0].equals("text")) {
            TextBuilder textbuilder = new TextBuilder();
            Director director = new Director(textbuilder);
            director.construct();
            String result = textbuilder.getTextResult();
            System.out.println(result);

        } else if (args[0].equals("html")) {
            HTMLBuilder htmlbuilder = new HTMLBuilder();
            Director director = new Director(htmlbuilder);
            director.construct();
            String filename = htmlbuilder.getHTMLResult();
            System.out.println(filename + "が作成されました");

        } else {
            usage();
            System.exit(0);
        }
    }
    public static void usage() {
        System.out.println("Usage: java Main plain  プレーンテキストで文書作成");
        System.out.println("Usage: java Main html   HTMLファイルで文書作成");
    }
}
```

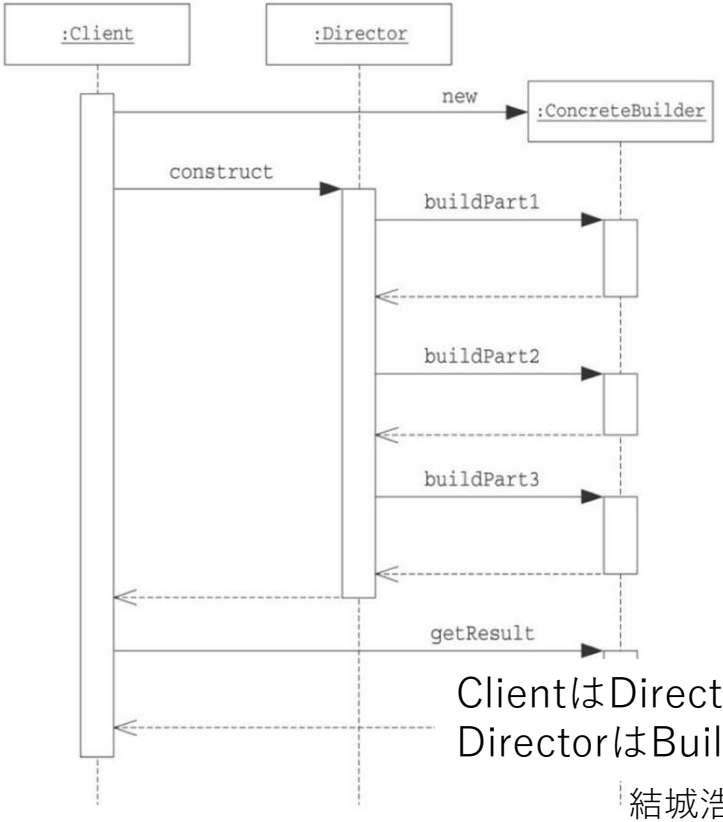
Builderの構築
Builderを持つDirectorの生成
Directorによる文書構築& 結果出力

利用側から見ると．．
文書構築自体はconstruct()の呼び出しのみで達成される．
細かな手順を気にする必要はない

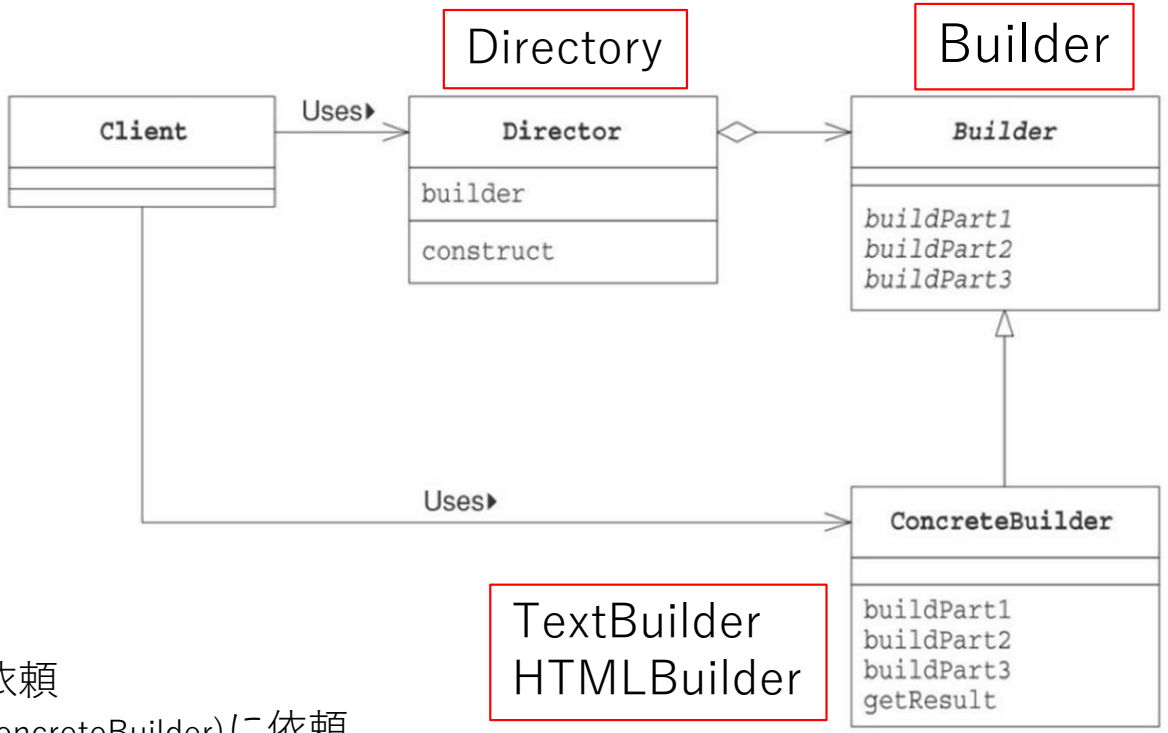
結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Builder パターンの登場人物

- Builder（建築者）の役
  - インスタンスを作成するためのインタフェースを定める
- ConcreteBuilder（具体的建築者）の役
  - Builder役のインタフェースを実装しているクラス
- Director（監督者）の役
  - Builder役のインタフェースを利用してインスタンスを生成する
    - ConcreteBuilderに依存したプログラミングは行わない
- Client（依頼者）の役：Builderパターンを利用する役



ClientはDirectorに依頼
DirectorはBuilder(ConcreteBuilder)に依頼



TextBuilder
HTMLBuilder

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考え方を広げるヒント

- 誰が何を知っているか？
  - オブジェクト指向では「誰が何を知っているか」は大切
  - どのクラスがどのメソッドを使えるかに注意してプログラムを作る
    - 例：MainクラスはBuilderクラスのメソッドは知らない
    - 例：Directorクラスも，具体的なBuilderのサブクラスには関知しない
  - 「知らないから入れ替えが可能」：「交換可能」＝部品としての価値が高い

- BuilderパターンとTemplate Methodパターンの比較
  - Builderパターン：Director役がBuilder役をコントロール
  - Template Methodパターン：スーパークラスがサブクラスをコントロール

演習2

- lec03_builder/*.java
- Builderを継承したLatexBuilderクラスを作成せよ
  - TextBuilderは，テキスト形式の文書を作成するクラス
  - HTMLBuilderは，HTML形式の文書を作成するクラス
  - 同様に，LatexBuilderは，Latex形式の文書を作成するクラス
    - 作成後，実行してみよう

  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec03_builder;

- （余力のある学生さん向け）前回扱ったTemplate Methodパターンによるプログラムを，Builderパターンを用いるように修正せよ
- （余力のある学生さん向け）Builderパターンを適用したプログラム例を作成せよ

- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

Factory Method パターン
～インスタンス生成をサブクラスにまかせる～

Factory Methodパターン

- Factory Method パターン：インスタンスの生成をサブクラスに任せる
  - インスタンス生成の枠組み（フレームワーク）と実際のインスタンス生成のクラスとを分けて考えるパターン
  - Template Methodパターンをインスタンス生成の場面に適用したもの
    - インスタンスの生成工場をTemplate Methodパターンで構成したもの
- スーパークラスの役割
  - インスタンスの作り方をスーパークラス側で定める
    - 具体的なクラス名までは定めない
- サブクラスの役割
  - 具体的な肉付けは，すべてサブクラスに任せる
- 2つ（以上）のパッケージを利用する
  - 加えて，総称型を利用する

役割分担に注目しよう

Template Methodでの役割分担も振り返ろう

メソッドのアクセス制御

	publicなしクラス	publicありクラス
private	クラス内でのみ使える	クラス内でのみ使える
なし	同じパッケージからのみ使える	同じパッケージからのみ使える
protected	同じパッケージからのみ使える	同じパッケージとサブクラスからのみ使える
public	同じパッケージからのみ使える	他のパッケージから使える

「同じパッケージのみから使える」
＝異なるパッケージからは使えない
※パッケージを分けることで，
アクセスを制限することができる

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

サンプルプログラム

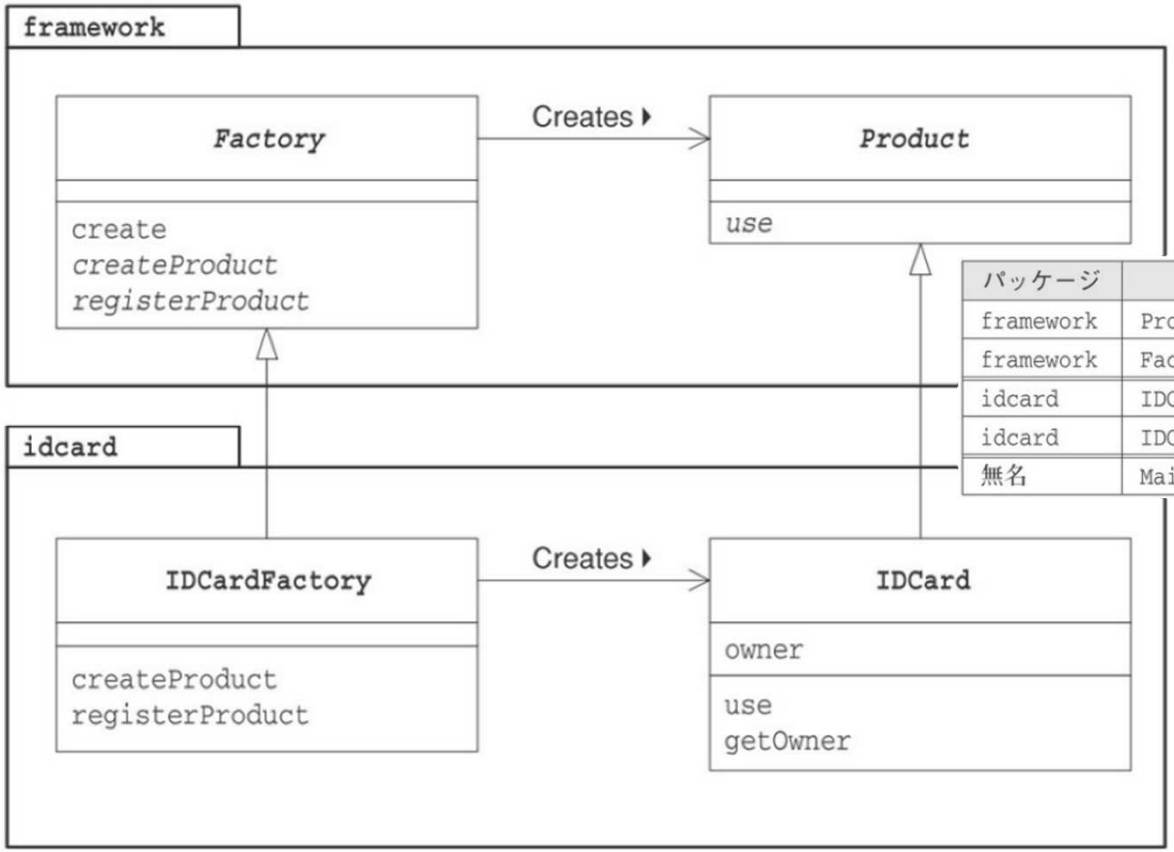
lec03_factory_method/*/*.java

コンパイル方法:
javac lec03_factory_method/Main.java
実行方法:
java lec03_factory_method /Main

- サンプルプログラム：「IDカード」を作る「工場」
- 2つのパッケージと4つのクラス
  - framework：インスタンス生成のフレームワークの側
    - Productクラス：「製品」を表現したクラス
    - Factoryクラス：「工場」を表現したクラス
  - idcard：肉付けを行っている側
    - ProductとFactoryのそれぞれを具体化

「IDカードを作って登録する」という一連の作業を行う

この一連の作業を
テンプレートメソッドで実現する



Factory（工場）でProduct（製品）を作る
製品生成において共通するプロセスを
テンプレートメソッドとして実装

パッケージ	名　前	解　説
framework	Product	抽象メソッドuseのみ定義されている抽象クラス
framework	Factory	メソッドcreateを実装している抽象クラス
idcard	IDCard	メソッドuseを実装しているクラス
idcard	IDCardFactory	メソッドcreateProduct,registerProductを実装しているクラス
無名	Main	動作テスト用のクラス

横のつながり：「工場」と「製品」
縦のつながり：
「フレームワーク」（抽象クラス）と
「肉付け」（実クラス）

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

frameworkパッケージ（枠組みを決める＝抽象クラス（・インタフェース）の利用）　**抽象クラス**

- Productクラス：「製品」を表現した抽象クラス
  - 抽象メソッド（use）を宣言している

```
public abstract class Product {
    public abstract void use();
}
```

- Factoryクラス：「工場」を表現した抽象クラス
  - テンプレートメソッド（create）を定義
    - createProductで製品を作り，registerProductで登録する
    - 具体的な実装はサブクラスに任せる

すべての工場（Factory）は，
createProduct（製品を作る）
registerProduct（製品を登録する）
メソッドを実装する必要がある

「Productを継承したT型」という意味（Tは後から決める）
T型に対する工場

```
public abstract class Factory<T extends Product> {
 抽象メソッドの宣言
    protected abstract T createProduct(String owner);
    protected abstract void registerProduct(T product);

    //生成において，Template Methodを使う
    public final T create(String owner){
        T p = createProduct(owner);
        registerProduct(p);
        return p;
    }
}
```

T createメソッドが
テンプレートメソッド

「作って」「登録」後，インスタンスを返す
具体メソッドの中で，抽象メソッドを呼び出す
（どう作る？どう登録する？はサブクラス任せ）

T型って？と思うかもしれませんが，
表面的な字面に意味はありません．
<T extends Product> なので，
Productの一種であるという意味のみです

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec03_factory_method/**idcard**/*.java

具体クラス

idcardパッケージ（肉付けを行う）

- IDCardクラス：認識番号カードを表すクラス
  - Productを継承＝＞抽象メソッドuseを実装
- IDCardFactoryクラス：IDCardの工場
  - Factoryを継承=>抽象メソッド を実装
  - createProduct : IDCardインスタンスの生成
  - reisterProduct：リストownerへの登録

コンストラクタ：
publicもprivateも付かない
=同一パッケージから利用可能
=異なるパッケージから利用不可
= 利用者はIDCardFactoryのcreateを通じてのみIDCardの生成が可能

```java
import lec03_factory_method.framework.Product;

public class IDCard extends Product{
    private String owner;
    IDCard(String owner){ //コンストラクタ
        System.out.println(owner + "のカードを作ります");
        this.owner = owner;
    }
    public void use(){
        System.out.println(owner + "のカードを使います");
    }
    public String getOwner(){
        return owner;
    }
}
```

抽象メソッドの実装

T = IDCardとする
Factoryを継承

```java
import lec03_factory_method.framework.Factory;

public class IDCardFactory extends Factory<IDCard> {
    private List<String> owners = new ArrayList<String>();

    protected IDCard createProduct(String owner) {
        return new IDCard(owner);
    }
```

作る：シンプルにnewしたものを返す

抽象メソッドを実装
テンプレートメソッドから利用

```java
    protected void registerProduct(IDCard product) {
        owners.add( product.getOwner() );
    }
```

登録：リストownersに追加する

```java
    public List<String> getOwners(){
        return owners;
    }
}
```

結城浩
『Java言語で学ぶデザインパターン入門第３版』
SBクリエイティブ(2021)より

lec03_factory_method/*/*.java

コンパイル方法: javac lec03_factory_method/Main.java
実行方法: java lec03_factory_method/Main

Mainクラス：カードの生成と利用

```
import lec03_factory_method.framework.Factory;
import lec03_factory_method.framework.Product;
import lec03_factory_method.idcard.IDCard;
import lec03_factory_method.idcard.IDCardFactory;

public class Main {
    public static void main(String[] args) {
        Factory<IDCard> factory = new IDCardFactory();
        Product card1 = factory.create("結城浩");
        Product card2 = factory.create("とむら");
        Product card3 = factory.create("佐藤花子");
        card1.use();
        card2.use();
        card3.use();
    }
}
```

実行結果

```
結城浩のカードを作ります
とむらのカードを作ります
佐藤花子のカードを作ります
結城浩のカードを使います
とむらのカードを使います
佐藤花子のカードを使います
```

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

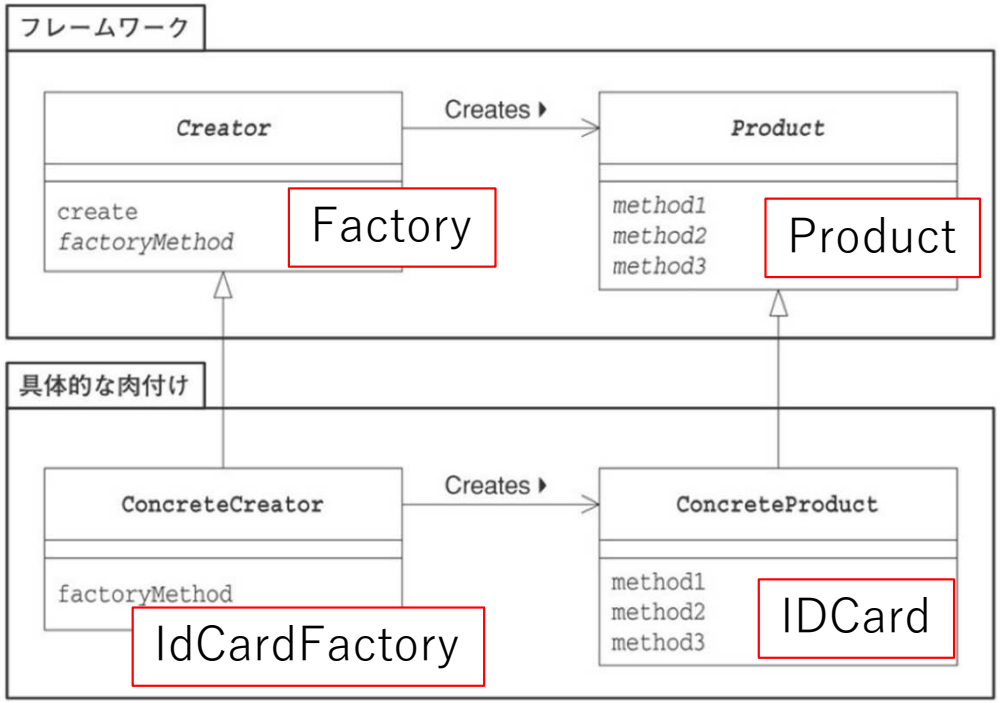
Factory Methodパターンの登場人物

フレームワーク側

- Product（製品）の役
  - 生成されるインスタンスがもつインタフェースを定める抽象クラス
- Creator（作成者）の役
  - Product役を生成する抽象クラス
  - 生成のためのテンプレートメソッドを定義
  - new によるインスタンスの生成を，インスタンス生成のためのメソッド呼び出しに変えることで，具体的なクラス名による束縛からスーパークラスを解放している

肉付けをする側

- ConcreteProduct（具体的製品）の役
  - 具体的な製品
- ConcreteCreator（具体的作成者）の役
  - 具体的な製品を生成する工場



Creator とConcreteCreatorの間に
Template Methodパターンが出現している

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるヒント

- フレームワークと肉付け
  - frameworkパッケージは，肉付けのパッケージに依存していない
  - frameworkパッケージの中身を修正せずに，全く別の「製品」「工場」を作ることが可能（「テレビ」と「テレビ工場」．．．などなど）
- インスタンス生成：メソッドの実装方法
  - FactoryクラスのcreateProductは抽象メソッド．実装方法は3通り

1. 抽象メソッドにする：サブクラスによるメソッドの実装が必須

```
abstract class Factory {
        public abstract Product createProduct(String name) ;
}
```

2. デフォルト実装の準備

```
class Factory{
        public Product createProduct(String name){ return new Product(name); }
}
```

   - Productを直接 new する．Productクラスの抽象クラス化は不可

3. エラーにする

   - throw new FactoryMethodRuntimeException(); //未学修内容

演習3

- lec03_factory_method/roomcard/*.java
- パッケージ idcard 参考に，ルームカード（RoomCard）とそれを生成する工場（RoomCardFactory）を表すプログラムを作成せよ
  - 各RoomCardのインスタンスには，所有者に加え，固有のIDを付与すること
    - 利用する（=use()を呼びだす）たびに，所有者と固有IDを表示すること
  - 各RoomCardのインスタンスに対し，その利用回数の上限を10回と設定する
    - → 利用回数（＝use()の呼び出し回数）が10回を超えた場合に，「利用上限を超えている旨を表示する」ように実装すること
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec03_factory_method/roomcard;
- （余力のある学生さん向け）Factory Method パターンを適用したプログラム例を作成せよ
- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください